

Docker Compose

- 여러 개의 명령어를 하나의 yaml파일로 정리해서, 한번에 시스템 전체를 실행하고 종료와 폐기까지 시키는 방식.
- Dockerfile과의 차이점
 - Dockerfile은 하나의 이미지를 만드는 방법이고, Docker Compose는 컨테이너+환경설정, 네트워크, 볼륨을 한 번에 만드는 방식이다.
 - Dockerfile에서는 네트워크와 볼륨 생성 및 설정이 불가능하다.
- 쿠버네티스와의 차이점
 - 쿠버네티스는 컨테이너를 관리하는 도구이지만, 도커 컴포즈는 컨테이너를 생성하고 삭제하는 것 뿐이다. (관리 불가능)

작성방법(yml)

컴포즈 파일의 항목

항목	docker run 커맨드의 해당 옵션 또는 인자	내용
version	-	도커 컴포즈의 버전을 기재한다.
services	-	컨테이너를 정의한다. (컨테이너의 집합체를 주로 서비스라고 한다.)
image	이미지 인자	사용할 이미지를 지정
networks	—net	접속할 네트워크를 지정
volumes	-v, —mount	스토리지 마운트(볼륨)를 설정
ports	-p	포트 설정
environment	-e	환경변수 설정
depends_on	없음	다른 서비스에 대한 의존관계를 정의
restart	없음	컨테이너 종료 시 재시작 여부를 설정

- depends_on은 다른 서비스에 대한 의존관계를 나타냄
 - 컨테이너를 생성하는 순서나 연동 여부를 정의
 - ex. penguin 컨테이너의 정의에 'depends_on: -namgeuk' 내용이 포함됐다면 namgeuk 컨테이너를 생성한 다음에 penguin 컨테이너를 만들 : 워드프레스처럼 MySQL 컨테이너가 먼저 있어야 하는 경우에 컨테이너 생성 순서를 지정
- restart는 컨테이너 종료 시 재시작 여부를 설정함

설정값	내용
no	재시작하지 않는다.
always	항상 재시작한다.
on-failure	프로세스가 0 외의 상태로 종료됐다면 재시작한다.
unless-stopped	종료 시 재시작하지 않음. 그 외에는 재시작한다.

예제

예제1

```
version: "3"

services:
  db:
    image: mysql:5.7
    restart: always
    ...
```

```
...  
command:  
  - --character-set-server=utf8mb4  
  - --collation-server=utf8mb4_unicode_ci  
volumes:  
  - ./database:/var/lib/mysql  
environment:  
  MYSQL_ROOT_PASSWORD: "root1234"  
  MYSQL_DATABASE: "examplesdb"  
  MYSQL_USER: "urstory"  
  MYSQL_PASSWORD: "u1234"  
ports:  
  - "3306:3306"
```

예제1 설명

- version: docker compose의 파일 버전
- services: 한개 또는 여러 개의 docker container를 설정
- image: docker container의 이름 정의
- restart: docker container가 다운되었을 경우, 항상 재시작하라는 설정
- volumes: docker run 명령의 -v 옵션과 동일한 역할
- environment: docker run 명령의 -v 옵션과 동일한 역할

예제2

```
version: "3"

services:
  app:
    build:
      context: ./01_FLASK_DOCKER
      dockerfile: Dockerfile
    ports:
      - "80:8080"
    container_name: appcontainer
    depends_on:
      - db
  ...
```

```
...
db:
  image: mysql:5.7
  restart: always
  command:
    - --character-set-server=utf8mb4
    - --collation-server=utf8mb4_unicode_ci
  volumes:
    - ./mysqldata:/var/lib/mysql
  environment:
    - MYSQL_ROOT_PASSWORD=root1234
    - MYSQL_DATABASE=examplesdb
  ports:
    - "3306:3306"
  container_name: dbcontainer
```

예제2 설명

- ports: docker run 명령의 -p 옵션과 동일한 역할
- networks: docker container 간의 네트워크 분리를 위해 추가로 설정을 한다.
- build: docker image를 Dockerfile 기반으로 작성 시 사용한다.
 - 참고로, context는 Dockerfile이 있는 디렉토리를 의미하며 dockerfile은 Dockerfile의 파일명을 의미한다.
- links: docker run 명령의 -link 옵션과 유사한 기능
 - 기본적으로 docker-compose 내부에선 모든 컨테이너가 소통할 수 있기 때문에 사용되지 않음.
- container_name: docker container를 만들 때, 해당 컨테이너 이름을 설정할 수 있다.
- depends_on: depend_on을 사용하여 컨테이너가 실행되는 순서를 제어할 수 있다.

실행방법

#컨테이너, 볼륨, 네트워크를 생성하고 실행
docker-compse up {옵션값}

up의 옵션 (컨테이너, 네트워크, 볼륨을 생성)

옵션	내용
-d	백그라운드로 실행
--force-recreate	설정이나 이미지가 변경되지 않더라도 컨테이너를 무조건 재생성
--no-recreate	컨테이너가 존재할 경우, 다시 생성하지 않음
--no-build	이미지가 없어도 이미지를 빌드하지 않음
--build	컨테이너를 실행하기 전에 이미지를 빌드
--abort-on-container-exit	컨테이너가 하나라도 종료되면 모든 컨테이너를 종료
-t, --timeout	컨테이너의 종류 타임아웃 설정. 디폴트 10초
--remove-orphans	컴포즈 파일에 정의되지 않은 컨테이너를 삭제
--scale	스케일링(컨테이너를 여러개 실행)

#컨테이너와 네트워크를 종료 및 삭제. 볼륨과 이미지는 삭제되지 않는다.
docker-compose down {옵션}

down의 옵션 (컨테이너와 네트워크를 삭제)

옵션	내용
--rmi 옵션	컴포즈 종료 시, 이미지도 삭제. all로 하면 모든 이미지 삭제, local로 하면 태그가 없는 이미지만 삭제
-v, --volumes	volumes 항목의 볼륨들을 삭제. external 볼륨은 삭제하지 않음.
--remove-orphans	컴포즈 파일에 정의되지 않은 서비스의 컨테이너도 모두 삭제